

```
"""
```

```
verrouillage_3_3_confinement.py  
=====
```

Objectif : verrouiller le point 3.3 de l'audit V107+1.

Démontrer la suppression statistique des triangles "isolés" (n'appartenant à aucun K4) dans la mesure de Gibbs canonique – analogue du confinement.

Définition opérationnelle (V107+1) :

Un triangle $T = \{i, j, k\} \subset V$ est dit "déconfiné" s'il n'existe aucun 4-clique K4 contenant T (i.e. $\{i, j, k, l\}$ n'est pas un 4-clique pour aucun l).

Prédiction du framework :

Dans la mesure de Gibbs canonique ($\beta=0.30$, $\tau=0.50$), la fraction de triangles déconfinés est statistiquement SUPPRIMÉE par rapport au baseline Erdős-Rényi à même densité d'arêtes.

Méthode (Rasoir d'Ockham, 3 chemins) :

Chemin 1 – Mesure directe sur configurations Blob à l'équilibre :

$f_{\text{déconf}}^{\text{Blob}} = (n_{\text{tri}} - n_{\text{tri_in_K4}}) / n_{\text{tri}}$

Chemin 2 – Baseline Erdős-Rényi à même densité d'arêtes :

$f_{\text{déconf}}^{\text{ER}}$ mesurée sur graphes ER tirés au hasard

Chemin 3 – Calcul Boltzmann analytique du ratio des poids

$P(\text{triangle isolé}) / P(\text{triangle dans K4})$ sous π_N

```
"""
```

```
import numpy as np
```

```
import math
```

```
import time
```

```
import json
```

```
from collections import deque
```

```
from scipy.spatial import KDTree
```

```
from scipy.stats import ttest_ind
```

```
KAPPA_0 = 8
```

```
BETA = 0.30
```

```
TAU = 0.50
```

```
N_TEST = 200
```

```
N_SEEDS = 5
```

```
# Pipeline minimal
```

```
def _rips_radius(N, kappa=KAPPA_0): return (kappa / ((N-1) * (4/3) * math.pi)) **
```

```
def _E0(N, kappa=KAPPA_0): return int(N * kappa / 2)
```

```

def _lcc_size(adj, N):
    visited = [False]*N; best = 0
    for s in range(N):
        if not visited[s]:
            sz = 0; q = deque([s]); visited[s] = True
            while q:
                v = q.popleft(); sz += 1
                for w in adj[v]:
                    if not visited[w]: visited[w] = True; q.append(w)
            best = max(best, sz)
    return best

def _build_initial(N, seed):
    r = _rips_radius(N); r_p = 1.5 * r
    E0 = _E0(N)
    rng = np.random.default_rng(seed)
    pts = rng.uniform(0.0, 1.0, (N, 3))
    tree = KDTree(pts)
    adj = [set() for _ in range(N)]
    for i, j in tree.query_pairs(r):
        adj[i].add(j); adj[j].add(i)
    pool = list(tree.query_pairs(r_p))
    rng2 = np.random.default_rng(seed + 50_000)
    K = sum(len(adj[i]) for i in range(N)) // 2
    att = 0; max_att = 10 * abs(E0 - K) + 5000
    while K != E0 and att < max_att:
        att += 1
        if K < E0:
            e = pool[rng2.integers(len(pool))]; u, v = e
            if v not in adj[u]: adj[u].add(v); adj[v].add(u); K += 1
        else:
            edges = [(i,j) for i in range(N) for j in adj[i] if j>i]
            if not edges: break
            e = edges[rng2.integers(len(edges))]; u, v = e
            adj[u].discard(v); adj[v].discard(u)
            if _lcc_size(adj, N) == N: K -= 1
            else: adj[u].add(v); adj[v].add(u)
    return pts, adj, pool, E0

def _count_triangles(adj, N):
    n = 0
    for i in range(N):
        for j in adj[i]:
            if j > i:
                n += len([k for k in adj[i] & adj[j] if k > j])
    return n

```

```

def compute_S(adj, n_tri, N):
    degs = np.array([len(adj[i]) for i in range(N)], float)
    D = degs.std() / (degs.mean() + 1e-8)
    return math.log(1 + n_tri) - BETA * D

def _fast_still_connected(adj, u, v, N):
    visited = {u}; q = deque([u])
    while q:
        node = q.popleft()
        for w in adj[node]:
            if w in visited: continue
            if node == u and w == v: continue
            if node == v and w == u: continue
            visited.add(w); q.append(w)
            if w == v: return True
    return False

def run_mcmc(pts, adj_init, pool, N, E0, seed):
    tau_eff = TAU / N
    rng = np.random.default_rng(seed + 200_000)
    adj = [set(a) for a in adj_init]
    edge_list = [(i,j) for i in range(N) for j in adj[i] if j>i]
    n_tri = _count_triangles(adj, N)
    S_curr = compute_S(adj, n_tri, N)
    n_steps = (100 + 200) * N
    u_arr = rng.random(size=n_steps)
    pool_arr = np.array(pool, dtype=np.int32)
    n_pool = len(pool_arr)
    for step in range(n_steps):
        if not edge_list: break
        idx_out = int(rng.integers(len(edge_list)))
        u, v = edge_list[idx_out]
        e_in = None
        for _ in range(30):
            idx_in = int(rng.integers(n_pool))
            a, b = int(pool_arr[idx_in,0]), int(pool_arr[idx_in,1])
            if b not in adj[a]: e_in = (a, b); break
        if e_in is None: continue
        a, b = e_in
        if len(adj[u]) <= 2 or len(adj[v]) <= 2:
            if not _fast_still_connected(adj, u, v, N): continue
        tri_lost = len(adj[u] & adj[v])
        adj[u].discard(v); adj[v].discard(u)
        adj[a].add(b); adj[b].add(a)
        tri_gained = len(adj[a] & adj[b])
        n_tri_new = n_tri - tri_lost + tri_gained
        S_new = compute_S(adj, n_tri_new, N)

```

```

    dS = S_new - S_curr
    if dS >= 0 or u_arr[step] < math.exp(dS / tau_eff):
        n_tri = n_tri_new; S_curr = S_new
        edge_list[idx_out] = (a, b)
    else:
        adj[u].add(v); adj[v].add(u); adj[a].discard(b); adj[b].discard(a)
return adj, n_tri

def count_triangles_and_deconfined(adj, N):
    """
    Compte les triangles et identifie ceux "déconfinés" (sans 4ème commun = pas d
    Renvoie (n_tri, n_deconfined, n_in_K4).
    """
    n_tri = 0
    n_deconfined = 0
    n_in_K4 = 0
    for i in range(N):
        Ni = adj[i]
        for j in Ni:
            if j <= i: continue
            Nj = adj[j]
            common_ij = Ni & Nj
            for k in common_ij:
                if k <= j: continue
                n_tri += 1
                # Triangle {i, j, k}. Pour être dans K4, besoin de  $1 \in N(i) \cap N(j)$ 
                common_ijk = common_ij & adj[k]
                if len(common_ijk) > 0:
                    n_in_K4 += 1
            else:
                n_deconfined += 1
    return n_tri, n_deconfined, n_in_K4

def make_er_graph_connected(N, E0, seed):
    """Génère un graphe ER (uniform random) à N sommets et E0 arêtes, et le rend
    rng = np.random.default_rng(seed + 999_000)
    all_pairs = [(i, j) for i in range(N) for j in range(i+1, N)]
    selected = rng.choice(len(all_pairs), size=E0, replace=False)
    adj = [set() for _ in range(N)]
    for idx in selected:
        u, v = all_pairs[idx]
        adj[u].add(v); adj[v].add(u)
    # Force connexité par rewiring glouton
    visited = [False]*N
    if _lcc_size(adj, N) < N:

```

```

components = []
for s in range(N):
    if not visited[s]:
        comp = []; q = deque([s]); visited[s] = True
        while q:
            v = q.popleft(); comp.append(v)
            for w in adj[v]:
                if not visited[w]: visited[w] = True; q.append(w)
        components.append(comp)
# Rewire : prendre une arête d'une composante et la déplacer vers une aut
for i in range(1, len(components)):
    # Trouver arête à supprimer dans une grosse composante
    big = components[0]
    for u in big:
        if len(adj[u]) >= 2:
            for v in list(adj[u]):
                if v in big and len(adj[v]) >= 2:
                    adj[u].discard(v); adj[v].discard(u)
                    # Connecter à la composante i
                    a = u; b = components[i][0]
                    adj[a].add(b); adj[b].add(a)
                    big.extend(components[i])
                    break
            else: continue
        break
return adj

# ===== Main =====

if __name__ == "__main__":
    print("=" * 70)
    print("VERROUILLAGE 3.3 – Confinement QCD (suppression triangles déconfinés)")
    print("=" * 70)
    print(f"\nN = {N_TEST}, paramètres canoniques  $\kappa_0$ ={KAPPA_0},  $\beta$ ={BETA},  $\tau$ ={TAU}")
    print(f"{N_SEEDS} seeds Blob + {N_SEEDS} seeds ER baseline\n")

    # Chemin 1 – Mesure Blob à l'équilibre
    print("[Chemin 1] Mesure Blob à l'équilibre (MCMC Phase A)")
    blob_results = []
    for seed in range(N_SEEDS):
        t0 = time.time()
        pts, adj_init, pool, E0 = _build_initial(N_TEST, seed)
        if _lcc_size(adj_init, N_TEST) / N_TEST < 0.9: continue
        adj, _ = run_mcmc(pts, adj_init, pool, N_TEST, E0, seed)
        n_tri, n_deconf, n_in_K4 = count_triangles_and_deconfined(adj, N_TEST)
        f_deconf = n_deconf / n_tri if n_tri > 0 else 0

```

```

elapsed = time.time() - t0
print(f" Seed {seed}: n_tri={n_tri:>5}, n_in_K4={n_in_K4:>5}, "
      f"n_deconf={n_deconf:>5}, f_déconf = {f_deconf*100:5.2f}% [{elapsed}
blob_results.append({'seed': seed, 'n_tri': n_tri, 'n_deconf': n_deconf,
                    'n_in_K4': n_in_K4, 'f_deconf': f_deconf})

# Chemin 2 – Baseline Erdős-Rényi à même E_0
print(f"\n[Chemin 2] Baseline Erdős-Rényi à même E_0 = {_E0(N_TEST)}")
er_results = []
for seed in range(N_SEEDS):
    t0 = time.time()
    adj_er = make_er_graph_connected(N_TEST, _E0(N_TEST), seed)
    n_tri, n_deconf, n_in_K4 = count_triangles_and_deconfined(adj_er, N_TEST)
    f_deconf = n_deconf / n_tri if n_tri > 0 else 0
    elapsed = time.time() - t0
    print(f" Seed {seed}: n_tri={n_tri:>5}, n_in_K4={n_in_K4:>5}, "
          f"n_deconf={n_deconf:>5}, f_déconf = {f_deconf*100:5.2f}% [{elapsed}
er_results.append({'seed': seed, 'n_tri': n_tri, 'n_deconf': n_deconf,
                  'n_in_K4': n_in_K4, 'f_deconf': f_deconf})

# Chemin 3 – Estimation Boltzmann analytique
print(f"\n[Chemin 3] Estimation analytique du ratio Boltzmann")
# Différence de S entre config A (triangle isolé) et B (triangle dans K4 = +3
#  $\Delta S = \log(1+n+3) - \log(1+n) - \beta \cdot \Delta D \approx \log((n+4)/(n+1))$ 
# Pour  $n \sim 1300$  (canonique),  $\Delta S \approx \log(1304/1301) \approx 2.3e-3$ 
# Poids relatif :  $\exp(N^2 \cdot \Delta S / \tau)$ 
n_tri_canon = np.mean([r['n_tri'] for r in blob_results])
dS_per_K4 = math.log((n_tri_canon + 4) / (n_tri_canon + 1))
weight_ratio = math.exp(N_TEST**2 * dS_per_K4 / TAU)
print(f" n_tri canonique  $\approx$  {n_tri_canon:.0f}")
print(f"  $\Delta S$  pour ajout d'un K4 (+3 triangles)  $\approx$  {dS_per_K4:.5f}")
print(f" Facteur de Boltzmann  $N^2 \cdot \Delta S / \tau \approx$  {N_TEST**2 * dS_per_K4 / TAU:.2f}")
print(f" Ratio de poids  $P(K4)/P(\text{no } K4) \approx \exp({N\_TEST**2 * dS\_per\_K4 / TAU:.2f})$ 

# Test statistique
blob_f = np.array([r['f_deconf'] for r in blob_results])
er_f = np.array([r['f_deconf'] for r in er_results])

print(f"\n{'='*70}")
print("VERDICT (Rasoir d'Ockham)")
print(f"{'='*70}")
print(f" Blob : f_déconf = {blob_f.mean()*100:.2f}%  $\pm$  {blob_f.std()*100:.2f}")
print(f" ER : f_déconf = {er_f.mean()*100:.2f}%  $\pm$  {er_f.std()*100:.2f}%")

if blob_f.std() > 0 and er_f.std() > 0:
    t_stat, p_val = ttest_ind(blob_f, er_f, equal_var=False)
    suppression = (er_f.mean() - blob_f.mean()) / er_f.mean() * 100 if er_f.m

```

```

print(f"\n  Suppression Blob vs ER : {suppression:+.1f}%")
print(f"  Test t de Welch : t = {t_stat:.2f}, p = {p_val:.4e}")

if blob_f.mean() < er_f.mean() and p_val < 0.05:
    print(f"\n  ✓ Chemin 1 vs Chemin 2 : Blob supprime significativement
else:
    print(f"\n  Δ Suppression non significative ou inverse.")

print(f"\n  Chemin 3 : facteur Boltzmann encourage massivement la formati
print(f"  ( $\exp(N^2 \cdot \Delta S / \tau)$ ) >> 1 à N grand) – cohérent avec l'observation Che

if blob_f.mean() < er_f.mean() and p_val < 0.05:
    print(f"\n  ⇒ 3 chemins convergent : Suppression statistique de tria
    print(f"  ⇒ Analogue du confinement QCD CONFIRMÉ dans le framework B
    print(f"  ⇒ Point 3.3 [H/argument qualitatif] → [T] partiel.")

# Sauvegarde
out = {
    'N': N_TEST, 'kappa_0': KAPPA_0, 'beta': BETA, 'tau': TAU,
    'blob_results': blob_results,
    'er_results': er_results,
    'blob_f_deconf_mean': float(blob_f.mean()),
    'blob_f_deconf_std': float(blob_f.std()),
    'er_f_deconf_mean': float(er_f.mean()),
    'er_f_deconf_std': float(er_f.std()),
    'suppression_pct': float(suppression) if 'suppression' in dir() else None
    'dS_per_K4_addition': float(dS_per_K4),
    'boltzmann_factor_log': float(N_TEST**2 * dS_per_K4 / TAU),
}
with open('/home/claude/verrouillage_3_3_results.json', 'w') as f:
    json.dump(out, f, indent=2)
print(f"\n  Sauvegarde : /home/claude/verrouillage_3_3_results.json")

```